

```
# =====
# ALHOWIFI SMART TRADING - Python / Streamlit
# يعمل محلياً على جهازك بدون قيود
# تشغيل: streamlit run alhowifi_trading.py
# =====

import math, warnings
warnings.filterwarnings("ignore")
import numpy as np
import pandas as pd
import yfinance as yf
import streamlit as st
import plotly.graph_objects as go
from plotly.subplots import make_subplots
from datetime import datetime

st.set_page_config(page_title="ALHOWIFI SMART TRADING", layout="wide", initial_si

st.markdown("""
<style>
.main-title{font-size:2.2rem;font-weight:900;letter-spacing:4px;color:#00f5ff;
  text-shadow:0 0 10px #00d4ff,0 0 20px #00d4ff;margin-bottom:0;}
.sub{font-size:.85rem;color:#64748b;margin-bottom:1rem;letter-spacing:2px;}
.card{padding:14px;border-radius:12px;border:1px solid rgba(255,255,255,.08);back
.call-card{padding:14px;border-radius:12px;border:1px solid rgba(0,255,136,.3);ba
.put-card{padding:14px;border-radius:12px;border:1px solid rgba(255,64,96,.3);bac
.wait-card{padding:14px;border-radius:12px;border:1px solid rgba(251,191,36,.3);b
.safe-card{padding:12px;border-radius:10px;border:1px solid rgba(0,255,136,.3);ba
.risk-card{padding:12px;border-radius:10px;border:1px solid rgba(251,191,36,.3);b
.trap-card{padding:12px;border-radius:10px;border:1px solid rgba(255,64,96,.3);ba
</style>
""", unsafe_allow_html=True)

# =====
# SECTORS & STOCKS
# =====

SECTORS = {
    "💻 تقنية": ["AAPL", "MSFT", "GOOGL", "ORCL", "XLK"],
    "⚡ AI/رقائق": ["NVDA", "AMD", "AVGO", "MU", "LRCX"],
    "🔒 سايبر/نمو": ["META", "NFLX", "CRWD", "PANW"],
    "🛒 استهلاكي": ["TSLA", "AMZN", "COST"],
    "💰 مالي": ["JPM", "GS", "XLF"],
    "🏥 صحة/طاقة": ["LLY", "XOM", "XLE", "XLV"],
    "🏭 صناعي": ["GE", "CAT", "GLD"],
```

```

    "مؤشرات": ["SPY", "QQQ"],
}

ALL_STOCKS = [s for lst in SECTORS.values() for s in lst]

TIMEFRAME_MAP = {
    "1لحظيm": {"interval": "1m", "period": "1d"},
    "5مضاربةm": {"interval": "5m", "period": "5d"},
    "15مضاربةm": {"interval": "15m", "period": "60d"},
    "1ساعةH": {"interval": "60m", "period": "180d"},
    "يومي": {"interval": "1d", "period": "1y"},
    "أسبوعي": {"interval": "1wk", "period": "5y"},
    "شهري": {"interval": "1mo", "period": "10y"},
}

ATR_MULT = {
    "1لحظيm": 0.8, "5مضاربةm": 1.0, "15مضاربةm": 1.2,
    "1ساعةH": 1.5, "3.8: شهري", "2.8: أسبوعي", "2.0: يومي",
}

HIGHER_TF = {
    "1m": "5m", "5m": "15m", "15m": "60m", "60m": "1d", "1d": "1wk", "1wk": "1mo", "1mo": None
}

HIGHER_PERIOD = {
    "5m": "5d", "15m": "60d", "60m": "180d", "1d": "1y", "1wk": "5y", "1mo": "10y"
}

# =====
# MATH
# =====

def ema(s, n): return s.ewm(span=n, adjust=False).mean()
def sma(s, n): return s.rolling(n).mean()
def wma(s, n):
    w = np.arange(1, n+1, dtype=float)
    return s.rolling(n).apply(lambda x: np.dot(x, w) / w.sum(), raw=True)
def hma(s, n):
    h, sq = max(int(n/2), 1), max(int(math.sqrt(n)), 1)
    return wma(2*wma(s, h) - wma(s, n), sq)
def rs14(s):
    d = s.diff(); g = d.clip(lower=0); l = -d.clip(upper=0)
    ag = g.ewm(alpha=1/14, adjust=False).mean()
    al = l.ewm(alpha=1/14, adjust=False).mean()
    return (100 - 100 / (1 + ag/al.replace(0, 1e-9))).fillna(50)
def macd_ind(s, f=5, sl=13, sig=6):
    line = ema(s, f) - ema(s, sl); signal = ema(line, sig)
    return line, signal, line - signal
def atr14(df):
    hl = df.High - df.Low
    hc = (df.High - df.Close.shift()).abs()
    lc = (df.Low - df.Close.shift()).abs()

```

```

        return pd.concat([hl, hc, lc], axis=1).max(axis=1).ewm(alpha=1/14, adjust=False).
def vwap(df):
    tp=(df.High+df.Low+df.Close)/3
    return (tp*df.Volume).cumsum()/(df.Volume.cumsum().replace(0,np.nan))
def bollinger(s,n=20,m=2):
    mid=sma(s,n); dev=s.rolling(n).std()
    return mid,mid+m*dev,mid-m*dev
def nadaraya(s,w=50,b=3):
    vals=s.values.astype(float); out=np.full_like(vals,np.nan)
    idx=np.arange(w); ctr=w-1
    ws_base=np.exp(-0.5*((idx-ctr)/b)**2); ws_base/=ws_base.sum()
    for i in range(w-1,len(vals)):
        out[i]=np.dot(vals[i-w+1:i+1],ws_base)
    return pd.Series(out,index=s.index)
def rvol(df,n=20):
    return df.Volume/(df.Volume.rolling(n).mean().replace(0,np.nan))

def prepare(df):
    df=df.copy().dropna()
    if df.empty: return df
    df["EMA200"]=ema(df.Close,200)
    df["RSI"]=rsi14(df.Close)
    df["MACD"],df["SIGNAL"],df["HIST"]=macd_ind(df.Close)
    df["ATR"]=atr14(df)
    df["RVOL"]=rvol(df)
    df["VWAP"]=vwap(df)
    df["HF"]=hma(df.Close,18)
    df["HS"]=hma(df.Close,55)
    df["HM"]=hma(df.Close,14)
    df["NWC"]=nadaraya(df.Close,50,3)
    df["NWU"]=df["NWC"]+(df["ATR"]*2.5)
    df["NWL"]=df["NWC"]-(df["ATR"]*2.5)
    df["BBM"],df["BBU"],df["BBL"]=bollinger(df.Close)
    return df

def hma_trend(df):
    if df.empty or df["HF"].isna().all(): return "غير واضح"
    r=df.iloc[-1]; a=r["ATR"] if not pd.isna(r["ATR"]) else 0.01
    if abs(r["HF"]-r["HS"])<a*0.05: return "عرضي"
    n=len(df)
    fs=(df["HF"].iloc[-1]-df["HF"].iloc[-4]) if n>4 else 0
    ss=(df["HS"].iloc[-1]-df["HS"].iloc[-4]) if n>4 else 0
    if r["HF"]>r["HS"] and fs>0 and ss>=0: return "صاعد"
    if r["HF"]<r["HS"] and fs<0 and ss<=0: return "هابط"
    return "عرضي"

```

```
# 12-CONDITION SIGNAL
```

```
#
```

```
def build_signal(df, trend, trendH, tf_label):
    r=df.iloc[-1]; p=df.iloc[-2] if len(df)>1 else r
    cv,a=float(r.Close),float(r.ATR) if not pd.isna(r.ATR) else float(r.Close)*0.
    ri,rV=float(r.RSI),float(r.RVOL) if not pd.isna(r.RVOL) else 1
    ml,ms=float(r.MACD),float(r.SIGNAL)
    vw=float(r.VWAP); e2=float(r.EMA200) if not pd.isna(r.EMA200) else cv
    nU,nL=float(r.NWU) if not pd.isna(r.NWU) else cv*1.05,float(r.NWL) if not pd.
    bU,bL=float(r.BBU) if not pd.isna(r.BBU) else cv*1.02,float(r.BBL) if not pd.
    sup=float(df.Low.tail(50).min()); res=float(df.High.tail(50).max())
    atr_avg=float(df.ATR.tail(20).mean())
    aV=cv>vw; aE=cv>e2; mU=ml>ms; mD=ml<ms
    rOK=not pd.isna(rV) and rV>1.2
    aOK=a>atr_avg*0.9
    HL=len(df)>=3 and df.Low.iloc[-1]>df.Low.iloc[-2]
    LH=len(df)>=3 and df.High.iloc[-1]<df.High.iloc[-2]
    pH=float(p.High); pL=float(p.Low)
    tHu=trendH in ["عرضي","صاعد"]; tHd=trendH in ["هابط","عرضي"]

    CC={
        "① الفريم الأعلى":aV,"① HMA صاعد":trend=="صاعد",
        "② RSI > 55":ri>55,"② MACD+":mU,"② Higher Low":HL or cv>pH,
        "③ RVOL>1.2":rOK,"⑥ ATR نشط":aOK,
        "④ منطقة دخول":cv>pH or cv<=(nL+a*0.35),
        "⑦ BB سليم":cv<=bU,
    }

    CP={
        "① الفريم الأعلى":not aV,"① HMA هابط":trend=="هابط",
        "② RSI < 45":ri<45,"② MACD-":mD,"② Lower High":LH or cv<pL,
        "③ RVOL>1.2":rOK,"⑥ ATR نشط":aOK,
        "④ منطقة دخول":cv<pL or cv>=(nU-a*0.35),
        "⑦ BB سليم":cv>=bL,
    }

    cs=int(sum(CC.values())/11*100); ps=int(sum(CP.values())/11*100)
    sig="انتظار"; st="WAIT"
    if cs>=73 and cs>ps: sig="كول"; st="CONFIRMED" if cs>=85 else "ARMED"
    elif ps>=73 and ps>cs: sig="بوت"; st="CONFIRMED" if ps>=85 else "ARMED"
    elif cs>=55 and cs>ps: sig="كول"; st="ARMED"
    elif ps>=55 and ps>cs: sig="بوت"; st="ARMED"

    mW=abs(ml-ms)<max(0.02,a*0.02)
    trap="SAFE"; tR=[]
    if sig=="كول":
        if df.High.iloc[-1]>res and cv<res: trap="TRAP"; tR.append("اختراق وهمي")
        if ri>=75: trap="RISK" if trap!="TRAP" else trap; tR.append("شبع شراء")
        if not aV: trap="RISK" if trap!="TRAP" else trap; tR.append("تحت VWAP")
```

```

        if mW: trap="RISK" if trap!="TRAP" else trap; tR.append("ضعيف MACD")
        if cv>bU: trap="RISK" if trap!="TRAP" else trap; tR.append("BB خارج")
    if sig=="🔴 بوت":
        if df.Low.iloc[-1]<sup and cv>sup: trap="TRAP"; tR.append("كسر وهمي")
        if ri<=25: trap="RISK" if trap!="TRAP" else trap; tR.append("RSI بيعاً شبع")
        if aV: trap="RISK" if trap!="TRAP" else trap; tR.append("VWAP فوق")
        if mW: trap="RISK" if trap!="TRAP" else trap; tR.append("ضعيف MACD")
        if cv<bL: trap="RISK" if trap!="TRAP" else trap; tR.append("BB خارج")

    if trap=="TRAP": sig="🟡 انتظار"; st="TRAP"
    mm=ATR_MULT.get(tf_label,1)
    tgt={"entry":cv,"stop":float("nan"),"t1":float("nan"),"t2":float("nan"),"t3":
    if sig=="🟢 كول":
        tgt={"entry":cv,"stop":min(cv-a*mm,vw-a*0.3),"t1":cv+a*mm,"t2":cv+a*mm*2,
    elif sig=="🔴 بوت":
        tgt={"entry":cv,"stop":max(cv+a*mm,vw+a*0.3),"t1":cv-a*mm,"t2":cv-a*mm*2,

    zone="منطقة طلب" if cv<=nL else "منطقة عرض" if cv>=nU else "وسط النطاق"
    cont="استمرار صاعد" if (aV and mU and ri>=55) else "ضعف" if (mD and ri<50)
    if sig=="🔴 بوت":
        cont="استمرار هابط" if (not aV and mD and ri<=45) else "ضعف" if (mU ar

    return dict(sig=sig,status=st,cs=cs,ps=ps,trap=trap,trapR=tR,
                CC=CC,CP=CP,trend=trend,trendH=trendH,zone=zone,cont=cont,
                rsi=ri,rvol=rv,atr=a,aboveVWAP=aV,aboveEMA=aE,
                macdHist=float(r.HIST),bbPos="BB فوق" if cv>bU else "BB تحت" if c
                price=cv,sup=sup,res=res,tgt=tgt)

# =====
# DATA CACHE
# =====
@st.cache_data(ttl=60)
def get_data(sym, interval, period):
    try:
        df=yf.download(sym,interval=interval,period=period,auto_adjust=False,prog
        if df is None or df.empty: return pd.DataFrame()
        return df.dropna()
    except: return pd.DataFrame()

def get_prepared(sym, interval, period):
    df=get_data(sym,interval,interval,period)
    if df.empty: return df
    return prepare(df)

def scan_sym(sym, tf_label):
    cfg=TIMEFRAME_MAP[tf_label]
    df=get_prepared(sym,cfg["interval"],cfg["period"])

```

```

if df.empty: return None
trend=hma_trend(df)
trendH="غير واضح"
hInt=HIGHER_TF.get(cfg["interval"])
if hInt:
    hPer=HIGHER_PERIOD.get(hInt,"1y")
    dfH=get_prepared(sym,hInt,hPer)
    if not dfH.empty: trendH=hma_trend(dfH)
s=build_signal(df,trend,trendH,tf_label)
r=df.iloc[-1]
prev_c=float(df.Close.iloc[-2]) if len(df)>1 else s["price"]
chg=(s["price"]-prev_c)/prev_c*100
return {**s,"sym":sym,"change":chg,"df":df}

# =====
# CHART
# =====

def make_chart(df, sym, tgt, sig):
    chart=df.tail(120).copy()
    fig=make_subplots(rows=3,cols=1,shared_xaxes=True,
                      vertical_spacing=0.03,row_heights=[0.62,0.18,0.20])
    fig.add_trace(go.Candlestick(x=chart.index,open=chart.Open,high=chart.High,
                                low=chart.Low,close=chart.Close,name="السعر"),r
    for col,name,color,dash in [
        ("VWAP","VWAP","#00d4ff","solid"),("EMA200","EMA200","#475569","dot"),
        ("HF","HMA Fast","#00ff88","solid"),("HS","HMA Slow","#ff8c42","solid"),
        ("NWC","Nadaraya","#a78bfa","dash"),("NWU","NW Upper","#a78bfa","dot"),
        ("NWL","NW Lower","#a78bfa","dot"),("BBU","BB Upper","#00d4ff","dot"),
        ("BBL","BB Lower","#00d4ff","dot"),
    ]:
        if col in chart.columns:
            fig.add_trace(go.Scatter(x=chart.index,y=chart[col],name=name,
                                    line=dict(color=color,dash=dash,width=1.2),opacity=0.8),row=1,col
            for lbl,val,color in [("دخول",tgt["entry"],"white"),("وقف",tgt["stop"],"#ff40
                                ("1%",tgt["t1"],"#00ff88"),("2%",tgt["t2"],"#00d4ff"),
            if val and not (isinstance(val,float) and math.isnan(val)):
                fig.add_hline(y=val,row=1,col=1,line_color=color,line_dash="dot",
                             annotation_text=f"{lbl} {val:.2f}",annotation_position=
            if "RSI" in chart.columns:
                fig.add_trace(go.Scatter(x=chart.index,y=chart.RSI,name="RSI",
                                         line=dict(color="#a78bfa",width=1.3)),row=2,col=1)
                fig.add_hline(y=70,row=2,col=1,line_color="#ff4060",line_dash="dot")
                fig.add_hline(y=30,row=2,col=1,line_color="#00ff88",line_dash="dot")
            if "MACD" in chart.columns:
                fig.add_trace(go.Scatter(x=chart.index,y=chart.MACD,name="MACD",
                                         line=dict(color="#00d4ff",width=1.2)),row=3,col=1)
                fig.add_trace(go.Scatter(x=chart.index,y=chart.SIGNAL,name="Signal",

```

```

        line=dict(color="#fbbf24",width=1.2)),row=3,col=1)
fig.add_trace(go.Bar(x=chart.index,y=chart.HIST,name="Hist",
                    marker_color=["rgba(0,255,136,.5)" if v>=0 else "rgba(255,64,96,.5)"
                    row=3,col=1)
fig.update_layout(title=f"{sym} - ALHOWIFI SMART TRADING",template="plotly_da
                    height=900,xaxis_rangeslider_visible=False,
                    legend=dict(orientation="h",yanchor="bottom",y=1.02))
return fig

# =====
# SIDEBAR
# =====
st.sidebar.title("🔍 الإعدادات")
mode=st.sidebar.radio(["مسح القائمة 🔍", "سهم واحد 🇮🇪"], "نوع العرض")
sym=st.sidebar.selectbox("الرمز", ALL_STOCKS, index=2)
tf_label=st.sidebar.selectbox("الفرم", list(TIMEFRAME_MAP.keys()), index=1)
st.sidebar.markdown("---")
st.sidebar.markdown("**القائمة بالقطاعات**")
for sec,stocks in SECTORS.items():
    st.sidebar.markdown(f"`{sec}`: {' '.join(stocks)}")

# =====
# HEADER
# =====
st.markdown('<div class="main-title">⚡ ALHOWIFI SMART TRADING</div>',unsafe_allow
st.markdown('<div class="sub">HMA · NADARAYA · ATR · VWAP · EMA200 · RSI · MACD ·

# =====
# SCAN MODE
# =====
if mode=="🔍 مسح القائمة":
    st.subheader(f"🔍 مسح {len(ALL_STOCKS)} سهم - {tf_label}")
    if st.button("▶ بدء المسح الكامل"):
        rows=[]
        pb=st.progress(0); status=st.empty()
        for i,s_ in enumerate(ALL_STOCKS):
            status.text(f"تحليل {s_}...")
            try:
                res=scan_sym(s_,tf_label)
                if res:
                    sec=[k for k,v in SECTORS.items() if s_ in v]
                    rows.append({"#":i+1,"القطاع":sec[0] if sec else "-", "الرمز":
                                "السعر":f"${res['price']:.2f}", "التغيير":f"${res['change']
                                "الإشارة":res["sig"], "الاتجاه":res["trend"],
                                "الأعلى":res["trendH"], "CALL%":res["cs"], "PUT%":res["ps"],
                                "Trap":res["trap"], "الدخول":f"${res['tgt']['entry']:.2f}"
                                "الوقف":f"${res['tgt']['stop']:.2f}" if not math.isnan(re

```

```

        "الهدف 1":f"${res['tgt']['t1']:.2f}" if not math.isnan(re
    except: pass
    pb.progress((i+1)/len(ALL_STOCKS))
    status.empty(); pb.empty()
    if rows:
        df_table=pd.DataFrame(rows).sort_values("CALL%",ascending=False)
        st.dataframe(df_table,use_container_width=True,hide_index=True)
    st.stop()

# =====
# SINGLE MODE
# =====

cfg=TIMEFRAME_MAP[tf_label]
with st.spinner(f"جاري تحليل {sym}..."):
    df=get_prepared(sym,cfg["interval"],cfg["period"])
    if df.empty:
        st.error(f"{sym} لا توجد بيانات لـ")
        st.stop()
    trend=hma_trend(df)
    trendH="غير واضح"
    hInt=HIGHER_TF.get(cfg["interval"])
    if hInt:
        hPer=HIGHER_PERIOD.get(hInt,"1y")
        dfH=get_prepared(sym,hInt,hPer)
        if not dfH.empty: trendH=hma_trend(dfH)
    s=build_signal(df,trend,trendH,tf_label)

r=df.iloc[-1]; prev_c=float(df.Close.iloc[-2]) if len(df)>1 else s["price"]
chg=(s["price"]-prev_c)/prev_c*100

# — TOP CARDS —
c1,c2,c3,c4,c5=st.columns(5)
with c1:
    st.markdown(f"""<div class="card">
    <div style="font-size:.8rem;color:#64748b;">الاتجاه الرئيسي</div>
    <div style="font-size:1.6rem;font-weight:900;color:{'#00ff88' if trend=='مساعد'
    <div style="font-size:.8rem;color:#64748b;">الفرم الأعلى: {trendH}</div>
    </div>""",unsafe_allow_html=True)
with c2:
    cls="call-card" if s["sig"]=="🟢 كول" else "put-card" if s["sig"]=="🔴 بوت"
    col_="green" if s["sig"]=="🟢 كول" else "red" if s["sig"]=="🔴 بوت" else "orange"
    st.markdown(f"""<div class="{cls}">
    <div style="font-size:.8rem;color:#64748b;">الإشارة</div>
    <div style="font-size:1.6rem;font-weight:900;">{s['sig']}</div>
    <div style="font-size:.8rem;color:#64748b;">{s['status']} · {s['cont']}</div>
    </div>""",unsafe_allow_html=True)
with c3:

```

```

cls="safe-card" if s["trap"]=="SAFE" else "risk-card" if s["trap"]=="RISK" el
tc="#00ff88" if s["trap"]=="SAFE" else "#fbbf24" if s["trap"]=="RISK" else "#
st.markdown(f""<div class="{cls}">
<div style="font-size:.8rem;color:#64748b;">Trap Filter</div>
<div style="font-size:1.6rem;font-weight:900;color:{tc};">{s['trap']}</div>
<div style="font-size:.8rem;color:#64748b;">{' | '.join(s['trapR']) if s['tra
</div>""",unsafe_allow_html=True)
with c4:
    st.markdown(f""<div class="card">
<div style="font-size:.8rem;color:#64748b;">القوة</div>
<div style="font-size:1.1rem;font-weight:700;color:#00ff88;">CALL {s['cs']}%<
<div style="font-size:1.1rem;font-weight:700;color:#ff4060;">PUT {s['ps']}%<
</div>""",unsafe_allow_html=True)
with c5:
    st.markdown(f""<div class="card">
<div style="font-size:.8rem;color:#64748b;">الموقع / RSI / RVOL</div>
<div style="font-size:1.1rem;font-weight:700;">{s['zone']}</div>
<div style="font-size:.8rem;color:#64748b;">RSI: {s['rsi']:.1f} | RVOL: {s['r
</div>""",unsafe_allow_html=True)

st.markdown("----")

# — CHART + PLAN —
col_chart,col_plan=st.columns([1.3,0.7])
with col_chart:
    st.plotly_chart(make_chart(df,sym,s["tgt"],s),use_container_width=True)

with col_plan:
    st.markdown(f"### {sym} — ${s['price']:.2f} ({chg:+.2f}%)")
    st.markdown(f"**الفرم: ** {tf_label} | **الاتجاه: ** {trend} | **الأعلى: ** {
    st.markdown("----")

t=s["tgt"]
if s["sig"]!="انتظار 🟡":
    st.markdown("**خطة الصفقة:**")
    cc1,cc2=st.columns(2)
    with cc1:
        st.metric("🟢 الدخول",f"${t['entry']:.2f}")
        st.metric("🎯 الهدف 1",f"${t['t1']:.2f}" if not math.isnan(t['t1']) e
        st.metric("🎯 الهدف 3",f"${t['t3']:.2f}" if not math.isnan(t['t3']) e
    with cc2:
        st.metric("🛑 الوقف",f"${t['stop']:.2f}" if not math.isnan(t['stop'])
        st.metric("🎯 الهدف 2",f"${t['t2']:.2f}" if not math.isnan(t['t2']) e
        if not math.isnan(t['stop']) and t['stop']!=t['entry']:
            rr=abs(t['t1']-t['entry'])/abs(t['entry']-t['stop'])
            st.metric("R:R",f"1:{rr:.1f}")

```

```

st.markdown("---")
st.markdown("**قراءة سريعة:**")
items=[
    ("الاتجاه",trend), ("الفرم الأعلى",trendH), ("الحالة",s["cont"]),
    ("VWAP","فوق  " if s["aboveVWAP"] else "تحت "),
    ("EMA200","فوق  " if s["aboveEMA"] else "تحت "),
    ("Nadaraya",s["zone"]), ("Bollinger",s["bbPos"]),
    ("MACD Hist",f"{s['macdHist']:.4f}"),
]
for k,v in items:
    st.write(f"- **{k}**: ** {v}")

st.markdown("---")
st.markdown("**الشروط المتحققة:**")
chks=s["CC"] if s["sig"]==" كول" else s["CP"]
passed=[k for k,v in chks.items() if v]
failed=[k for k,v in chks.items() if not v]
if passed:
    for c_ in passed: st.write(f" {c_}")
if failed:
    st.markdown("**غير متحققة:**")
    for c_ in failed: st.write(f" {c_}")

st.markdown("---")
st.caption("ALHOWIFI SMART TRADING © 2025 · للأغراض التحليلية فقط")

```